HACK ARENA • PROBLEM ANALYSIS & EXECUTION BLUEPRINT

Containerized Audience Segmentation & Personalization Service

Comprehensive End-to-End Analysis, Architectural Specification, Evaluation Standards, Edge-Case Matrix, and Ready-to-Deploy GitHub Repository Blueprint.

Duration	Total Marks	Target Architecture	Core Algorithm
2h 30m Build + 30m Report	**100 Marks**	**Docker 3-Microservices**	**Unsupervised KMeans / GMM**

1. Executive Summary & Core Challenge

Modern OTT streaming platforms log millions of user interactions (watch time, session durations, genre preference distribution, completion rates). Raw logs cannot directly inform personalization rules. The objective is to build a **lightweight, deterministic, containerized audience intelligence service** that transforms tabular viewer logs into defensible behavioral clusters, exposes them via a production-grade REST API, and evaluates the entire lifecycle automatically.

Business Objective	Replace generic one-size-fits-all recommendations with automated behavioral cohorts such as intense binge-watchers, casual short-form viewers, and weekend binge explorers to drive retention and engagement.
Engineering Goal	A turnkey one-command (docker compose up --build) system connecting trainer, api, and evaluator microservices through shared Docker storage volumes, with deterministic seeding and independent evaluation.

Non-negotiable: Build the smallest system that proves the complete idea. Do not use LLMs, heavy transformers, paid cloud APIs, notebook-only submissions, or retraining during inference. The core clustering task remains purely unsupervised.

2. 3-Service Microservice Architecture

The system is strictly decoupled into three services communicating via volumes and HTTP networks:

Service	Role	Responsibilities
1. Trainer Service	Runs First	Loads user_activity.csv; sanitizes and scales features; computes optimal K; fits KMeans/GMM deterministically; maps centroids to labels; saves /models/model.pkl; exits cleanly.
2. API Service	Long Running	FastAPI/Uvicorn; loads model once; GET /health; POST /recommend; strict Pydantic validation; returns segment, distance and recommendations; Docker healthcheck.
3. Evaluator Service	Automated Verification	Waits for healthy API; submits standard profiles; executes 10 edge cases; calculates metrics; generates metrics.json; logs verification report.

3. Machine Learning & Behavioral Feature Engineering

Feature Selection Matrix

Feature Category	Raw Input / Extracted Feature	Transformation & Behavioral Purpose
Engagement	watch_time_hours, total_sessions	Log-scale or StandardScaler; distinguishes power binge-viewers from low-touch viewers.
Session Dynamics	avg_session_mins	Continuous numeric; isolates casual snackable viewers (<20m) from cinematic viewers (>90m).
Content Preferences	top_genres (list of strings)	Multi-Hot Encoding / TF-IDF genre frequency vector.
Pattern & Diversity	genre_diversity_score, weekend_ratio	Entropy of genre distribution; differentiates focused niche viewers from generalist explorers.

Optimal Cluster Selection

- **Silhouette Coefficient (s):** Target $s > 0.40$.
- **Inertia:** Elbow curve across $K \in [3, 8]$.
- **Cluster Balance Ratio:** No cluster should contain <5% or >60% of users.

Segment Naming & Recommendation Logic

Cluster Segment Name	Behavioral Signature	Recommendation Strategy
High-Engagement Action Viewers	High watch time (>30h/wk), session mins >80m, Action/Thriller dominance	Blockbuster action franchises, high-octane thriller series, 4K HDR releases.
Casual Short-Session Viewers	Session duration <25m, frequent visits, Comedy/Shorts	Sitcoms, stand-up clips, snackable web series, trending highlights.
Genre-Explorers	Genre diversity >0.7, medium-to-high watch time, varied categories	Curated indies, international cinema, docuseries.
Low-Activity Weekend Viewers	Watch time <5h, weekend concentrated, Family/Animation	Popular crowd-pleasers, top 10 trending movies, low-friction family viewing.

4. Production API Specification

POST /recommend Request

```
{
  "user_id": "USR-8192",
  "watch_time_hours": 32.5,
  "top_genres": ["Action", "Thriller"],
  "avg_session_mins": 85.0
}
```

POST /recommend Response (200 OK)

```
{
  "user_id": "USR-8192",
  "segment_id": 3,
  "segment_name": "High-Engagement Action Viewers",
  "recommendations": ["The Dark Knight", "Inception", "Heat"
  "distance_to_centroid": 0.42
}
```

```
GET /health (200 OK)
{
  "status": "ok",
  "model_loaded": true,
  "version": "1.0.0",
  "n_clusters": 4
}
```

5. Evaluator Service & Edge-Case Testing Matrix

The evaluator must run automated integration tests covering the 10 specific edge cases required by the competition:

Scenario	Input	Expected Handling
1. Unknown / Unseen Genre	{"top_genres":["SpaceOpera","UnknownXYZ"]}	Ignore unknown genres or map to fallback; 200 OK.
2. Empty Genre List	{"top_genres":[]}	Zero genre vector; cluster by watch time/session metrics; 200 OK.
3. Zero Watch Time	{"watch_time_hours":0.0}	Valid lower bound; Low-Activity cluster; 200 OK.
4. Extreme Numeric Outlier	{"watch_time_hours":999999.0}	Clip to 99th percentile scaler ceiling; prevent overflow; 200 OK.
5. Missing Required Field	{"user_id":"USR-1"}	Pydantic validation rejects payload; 422 or 400.
6. String in Numeric Field	{"watch_time_hours":"twenty"}	Reject non-numeric type; 422/400.
7. Negative Numeric Values	{"avg_session_mins":-15.0}	ge=0 validator catches negative values; 422/400.
8. Idempotency Check	Same profile 5 times	100% deterministic output; 200 OK.
9. Request Before Model Loads	Before /models/model.pkl loaded	503 Service Unavailable; no crash.
10. Cold Deployment / Missing Volume	Empty /models/	Health reports degraded/model_loaded=false; evaluator retries.

Required Output: metrics.json


```
{
  "timestamp": "2026-10-05T18:30:00Z",
  "clustering_metrics": {"optimal_k": 4, "silhouette_score": 0.482, "inertia": 1243.82},
  "api_health_metrics": {"health_check_status": 200, "avg_response_time_ms": 4.2, "p99_response_time_ms": 11.8},
  "edge_case_results": {"total_tested": 10, "passed": 10, "failed": 0}
}
```

6. Marking Scheme Breakdown (100 Marks Total)

Component	Marks	Key Criteria
docker compose up clean first run	15	Zero prompts, zero manual copies, automatic directory creation, deterministic boot.
3-Service Architecture & Volume Sharing	10	trainer, api, evaluator using named volumes.
Healthcheck & Startup Readiness Ordering	8	depends_on conditions for trainer and API.
Container Hygiene & Security	7	python:3.11-slim, non-root execution, pinned requirements.
API Correctness & Validation	10	FastAPI Pydantic validators and sanitized errors.
Evaluator Correctness & metrics.json	10	Independent validation suite and valid metrics.json.
Segmentation Quality & Cluster Interpretation	10	Silhouette + Elbow justification and meaningful labels.
Reproducibility Across Clean Runs	5	random_state=42 and standard seeds.
REPORT.md Documentation	25	All 13 mandatory sections with honest edge-case reporting.
TOTAL	100	Full-marks target achievable with standardized boilerplate.

7. Suggested 2.5-Hour Execution Plan

Time	Task	Focus
00:00 - 00:20	Data & Feature Pipeline	Inspect CSV, handle nulls, define numerical and one-hot genre encoders.
00:20 - 00:55	Clustering & Persistence	KMeans silhouette tuning, labels, serialize pipeline to /models.
00:55 - 01:30	FastAPI & Recommendation	POST /recommend, GET /health, schemas, recommendation catalog.
01:30 - 02:00	Docker & Orchestration	Lean Dockerfiles, non-root users, volumes, healthcheck, compose.
02:00 - 02:30	Evaluator & Edge Cases	Automated 10-case harness and metrics.json.
02:30 - 03:00	REPORT.md & Polish	Documentation, cold-start verification, screenshots/logs, packaging.

8. GitHub Repository Blueprint & Directory Layout

Recommended Repository Link / Template: https://github.com//containerized-audience-segmentation-service

```
containerized-audience-segmentation-service/
├── docker-compose.yml
├── Makefile
├── README.md
├── REPORT.md
├── shared_models/
├── data/
│   └── user_activity.csv
├── trainer/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── train.py
├── api/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py
│   └── recommender.py
└── evaluator/
    ├── Dockerfile
    ├── requirements.txt
    ├── evaluate.py
    └── test_cases.json
```

Complete docker-compose.yml Specification

```yaml
version: "3.8"

services:
  trainer:
    build: {context: ./trainer, dockerfile: Dockerfile}
    volumes:
      - model_volume:/models
      - ./data:/data:ro
    environment:
      - RANDOM_SEED=42
      - MODEL_PATH=/models/clustering_pipeline.joblib
      - METADATA_PATH=/models/metadata.json
    restart: "no"

  api:
    build: {context: ./api, dockerfile: Dockerfile}
    ports: ["8000:8000"]
    volumes:
      - model_volume:/models:ro
    depends_on:
      trainer:
        condition: service_completed_successfully
    healthcheck:
      test: ["CMD-SHELL", "curl -f http://localhost:8000/health || exit 1"]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 5s
    restart: unless-stopped

  evaluator:
    build: {context: ./evaluator, dockerfile: Dockerfile}
    volumes:
      - ./metrics.json:/app/metrics.json
    environment:
      - API_URL=http://api:8000
    depends_on:
      api:
        condition: service_healthy
    restart: "no"

volumes:
  model_volume:
    driver: local
```

Git Initialization & Push Commands

```bash
git init -b main
git add .
git commit -m "feat: complete containerized audience segmentation & personalization service"
git remote add origin https://github.com/<your-username>/containerized-audience-segmentation-service.git
```

```
git push -u origin main
```

Generated for Hack Arena Student Hackathon • Ready for PDF submission